

PROJECT DOCUMENTATION

Product CRUD App

React · Node.js · Express · MySQL

A page-by-page walkthrough with screenshots

Prepared: 22 September 2026

Repository: [gaurav-softmate-crud-assignment](#)

A walkthrough of every page in the app, with a screenshot and an explanation of what the page does.

All screenshots were taken from the running app (frontend on port 5173, backend on port 5000) with sample data.

Contents

1. [What the app is](#)
 2. [How it works](#)
 3. [Page 1 — Login](#)
 4. [Page 2 — Register](#)
 5. [Page 3 — Product List](#)
 6. [Page 4 — Add Product](#)
 7. [Page 5 — Edit Product](#)
 8. [Page 6 — Product View](#)
 9. [API reference](#)
 10. [Database tables](#)
-

What the app is

A product manager. A user makes an account, logs in, and can add, view, edit and delete their products. Each product can have a picture.

Frontend: React 19, Vite, React Router, Axios **Backend:** Node.js, Express 5, MySQL, JWT, bcrypt, multer

How it works

A user logs in. The backend checks the password and sends back a **token**. The frontend saves that token in the browser and attaches it to every later request. The backend checks the token before touching the database.

```
Browser -> React page -> service -> axios (adds token)
                                   |
                                   v
                               Express route -> auth check -> controller -> MySQL
```

If the token is missing or expired, the backend replies **401** and the frontend sends the user back to the login page automatically.

There are 6 pages. 2 are open to everyone, 4 need a login.

Page	Address	Login needed
Login	/login	No
Register	/register	No
Product List	/products	Yes
Add Product	/products/add	Yes
Edit Product	/products/edit/:id	Yes
Product View	/products/:id	Yes

Page 1 — Login

The screenshot shows a web application interface for a login page. At the top left, there is a shopping cart icon followed by the text 'Product App'. At the top right, there are links for 'Login' and 'Register'. The main area of the page is a light gray rectangle containing a white login form. The form is titled 'Welcome Back' and has a subtitle 'Login to manage your products'. It contains two input fields: 'Email' and 'Password'. Below the 'Password' field is a blue 'Login' button. At the bottom of the form, there is a link that says 'Don't have an account? [Register](#)'.

The first page a user sees. They type their email and password to get in.

What it does

- Sends email and password to `POST /api/auth/login`
- Saves the returned token and the user's name in the browser
- Sends the user to the Product List page

Checks before sending

Field	Rule
Email	Must not be empty. Must contain @ .
Password	Must not be empty.

File: `frontend/src/pages/auth/Login.jsx`

When fields are empty

 **Product App**

LoginRegister

Welcome Back

Login to manage your products

Email

Email is required

Password

Password is required

Login

Don't have an account? [Register](#)

Nothing is sent to the server. The page shows red messages under each empty box. This check runs in the browser, so it is instant.

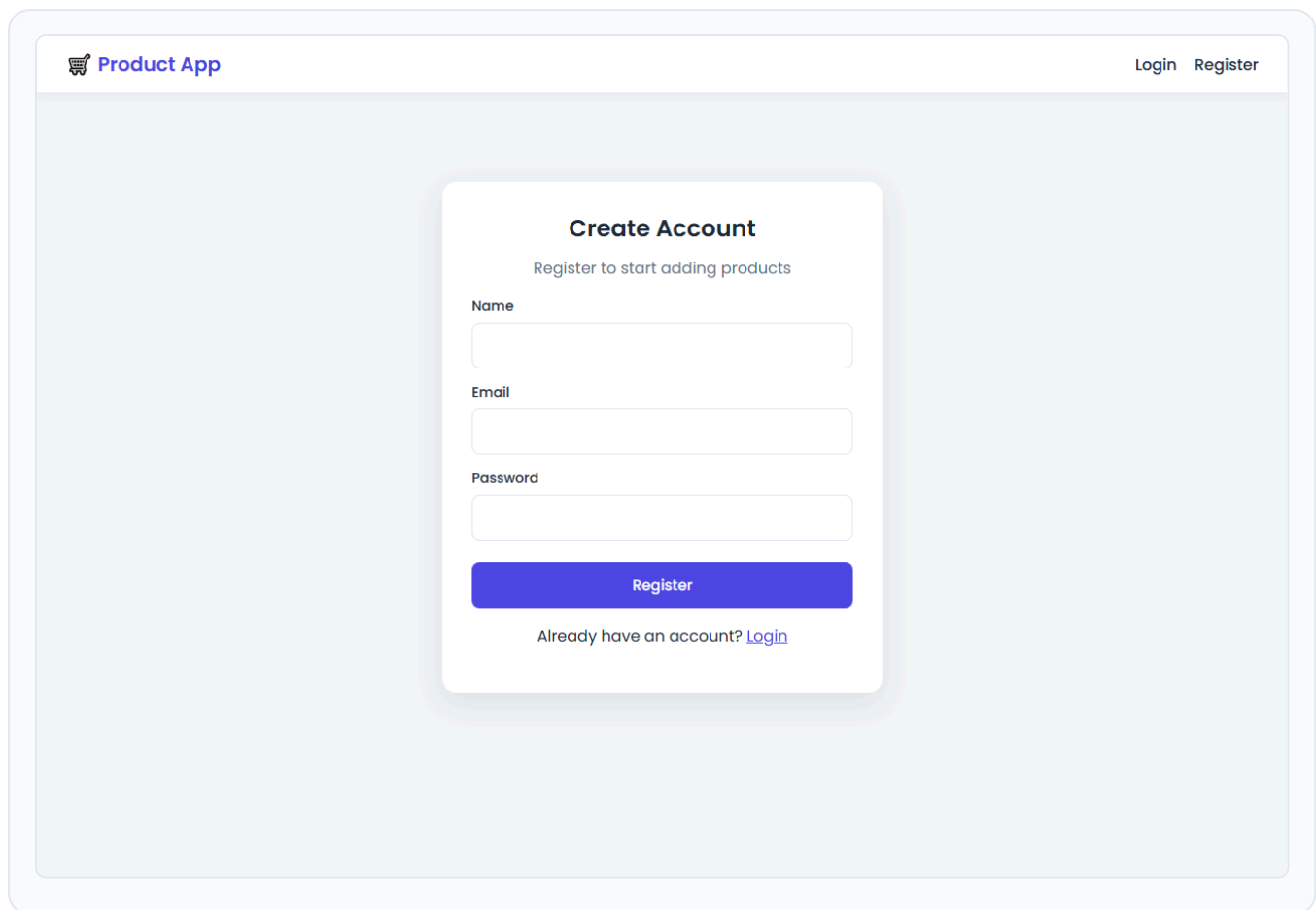
When the password is wrong

The screenshot shows a web application interface for a 'Product App'. At the top left is a shopping cart icon and the text 'Product App'. At the top right are links for 'Login' and 'Register'. The main content area features a white login card with a blue shadow. The card has the heading 'Welcome Back' and the subtext 'Login to manage your products'. A pink error bar with the text 'Invalid email or password' is displayed above the input fields. The 'Email' field contains 'test@example.com' and the 'Password' field is masked with dots. A blue 'Login' button is at the bottom of the form, followed by the text 'Don't have an account? [Register](#)'.

Here the request does go to the server. The server says no, and the page shows a pink error bar: **Invalid email or password**.

The message is deliberately vague. It does not say whether the email exists, so nobody can use this page to find out which emails are registered.

Page 2 — Register



The screenshot shows a web application interface for a 'Product App'. At the top left is a shopping cart icon and the text 'Product App'. At the top right are links for 'Login' and 'Register'. The main content area features a centered 'Create Account' form. The form has a title 'Create Account', a subtitle 'Register to start adding products', and three input fields labeled 'Name', 'Email', and 'Password'. Below these fields is a blue 'Register' button. At the bottom of the form, it says 'Already have an account? [Login](#)'.

Makes a new account. Needed on a fresh install, because the database starts with no users.

What it does

- Sends name, email and password to `POST /api/auth/register`
- The backend hashes the password with bcrypt before saving it
- On success, sends the user to the Login page

Checks before sending

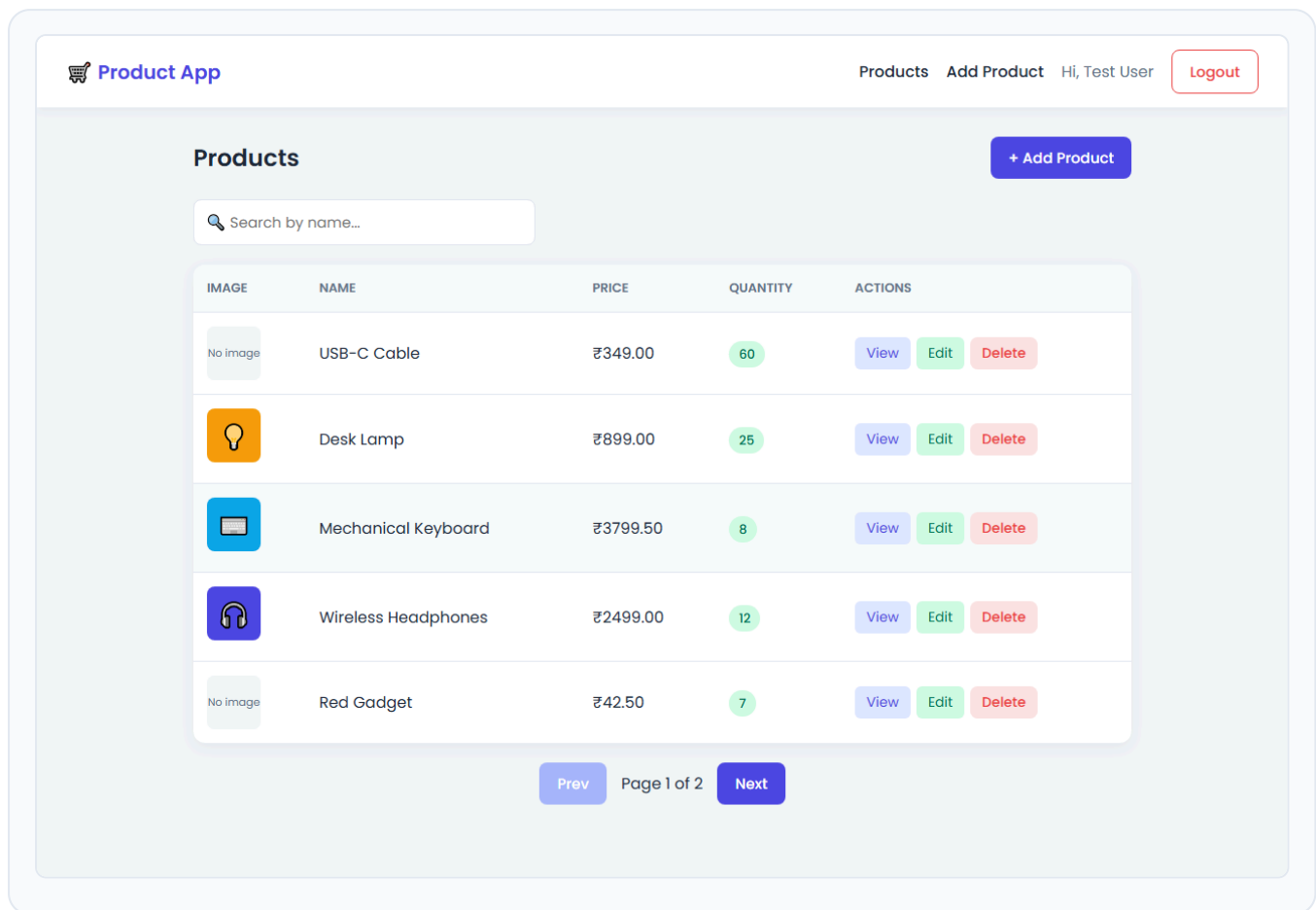
Field	Rule
Name	Must not be empty.
Email	Must not be empty. Must contain @ .
Password	At least 6 characters.

The same rules are checked again on the server in `backend/utils/validators.js`. The browser check is for speed; the server check is the real one, because anyone can skip the browser.

The email column is `UNIQUE` in the database, so the same email cannot register twice.

File: `frontend/src/pages/auth/Register.jsx`

Page 3 — Product List



The main page. It shows all products in a table: picture, name, price, quantity, and three buttons.

What it does

- Loads products from `GET /api/products?page=1&search=`
- **View** opens the product page
- **Edit** opens the edit form
- **Delete** asks "Are you sure?" first, then deletes and reloads the list
- **+ Add Product** opens the add form

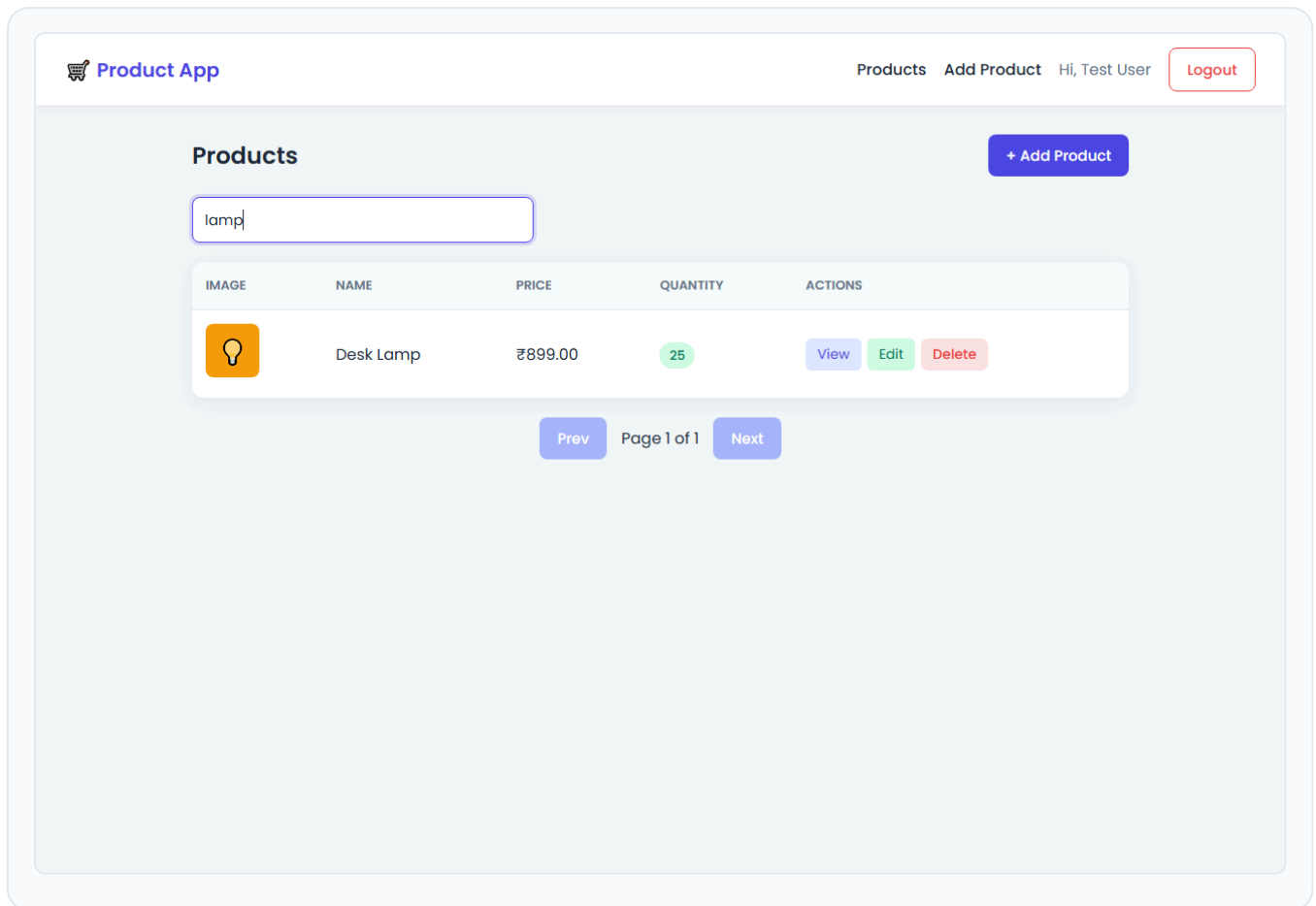
Notes on the screen

- Products with no picture show a grey **No image** box
- Quantity sits in a green pill
- The newest product is at the top (sorted by id, highest first)
- **5 products per page**. Here there are 6, so it says *Page 1 of 2*
- **Prev** is greyed out on page 1, **Next** is greyed out on the last page

While loading it shows *Loading...*. If nothing matches it shows *No products found*.

File: `frontend/src/pages/products/ProductList.jsx`

Search



Typing in the search box filters by name. Here "lamp" leaves only **Desk Lamp**, and the counter changes to *Page 1 of 1*.

The search runs on the server with SQL `LIKE`, not in the browser, so it searches all products and not only the 5 on screen. Typing also resets the page back to 1, so you never land on an empty page.

Page 4 — Add Product

Product App

Products Add Product Hi, Test User Logout

Add Product

Name

Description

Price

Quantity

Image

Choose File No file chosen

Save

A form to create a new product. Reached from the + **Add Product** button.

Fields

Field	Required	Notes
Name	Yes	
Description	No	Long text box
Price	Yes	Cannot be negative
Quantity	Yes	Cannot be negative
Image	No	Image files only, under 2MB

What it does

- Sends the form to `POST /api/products` as form-data (needed for the file)
- The picture is saved in `backend/uploads/` with a unique name
- On success, goes back to the Product List

File: `frontend/src/pages/products/ProductForm.jsx`

When fields are empty

The screenshot shows a web application titled "Product App" with a shopping cart icon. The navigation bar includes "Products", "Add Product", "Hi, Test User", and a "Logout" button. The main content area features a central "Add Product" form. The form has five input fields: "Name", "Description", "Price", "Quantity", and "Image". The "Name", "Price", and "Quantity" fields have red error messages below them: "Name is required", "Price is required", and "Quantity is required". The "Description" field is a text area, and the "Image" field is a file upload button labeled "Choose File" with the text "No file chosen". A blue "Save" button is at the bottom of the form.

Product App

Products Add Product Hi, Test User Logout

Add Product

Name

Name is required

Description

Price

Price is required

Quantity

Quantity is required

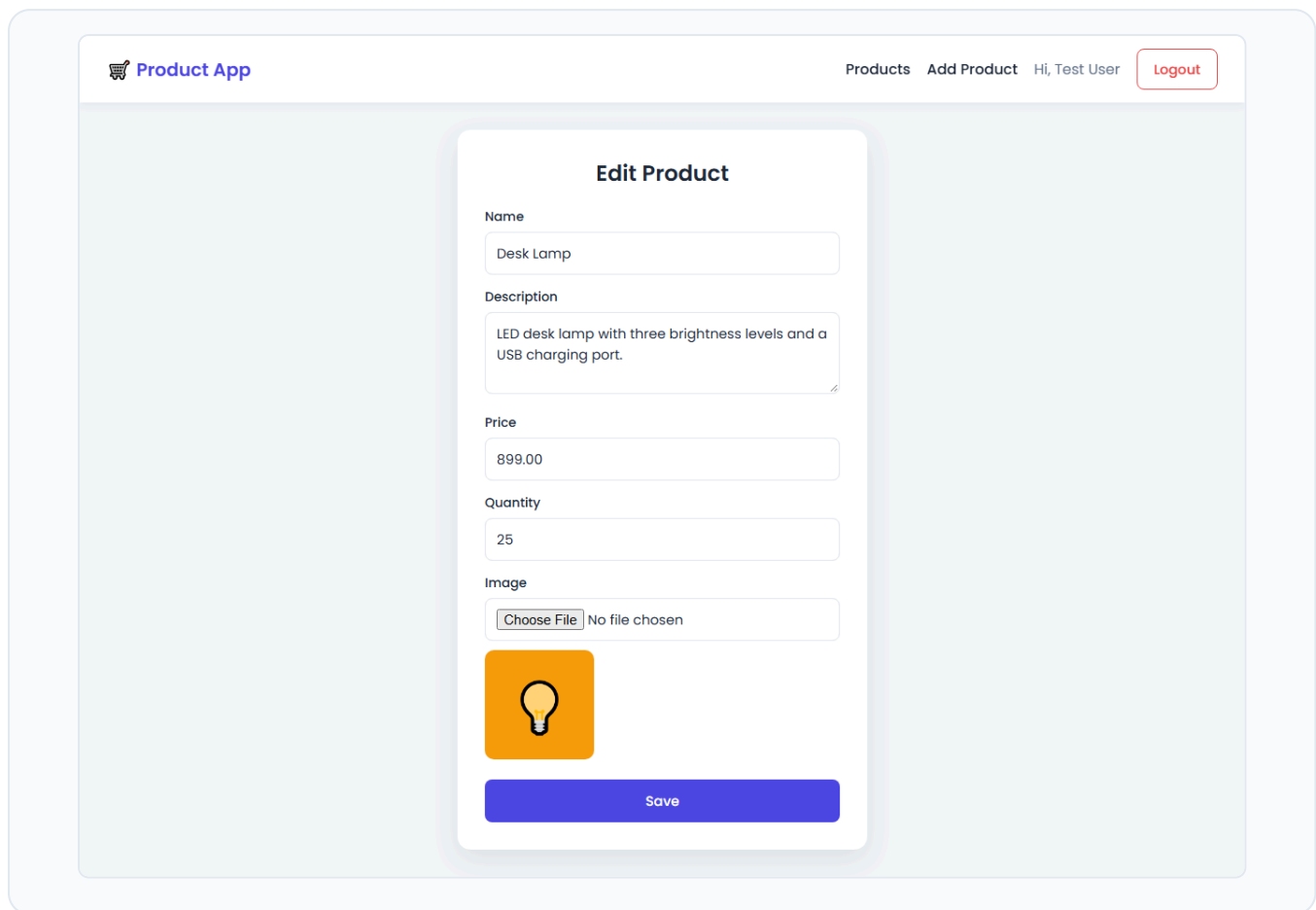
Image

Choose File No file chosen

Save

Red messages appear under Name, Price and Quantity. Description and Image are optional, so they stay clean.

Page 5 — Edit Product



The screenshot shows a web application interface for editing a product. At the top, there's a navigation bar with a shopping cart icon, the text 'Product App', and links for 'Products', 'Add Product', and a user profile 'Hi, Test User' with a 'Logout' button. The main content area is a light gray rectangle containing a white form titled 'Edit Product'. The form has five sections: 'Name' with a text input containing 'Desk Lamp'; 'Description' with a text area containing 'LED desk lamp with three brightness levels and a USB charging port.'; 'Price' with a text input containing '899.00'; 'Quantity' with a text input containing '25'; and 'Image' with a 'Choose File' button and a placeholder image of a lightbulb. A blue 'Save' button is at the bottom of the form.

The same form as Add, but the boxes are already filled with the product's current values. The title changes to **Edit Product**.

The app knows which mode it is in from the address. `/products/add` has no id, so it is a new product. `/products/edit/5` has an id, so it loads product 5 first and fills the form.

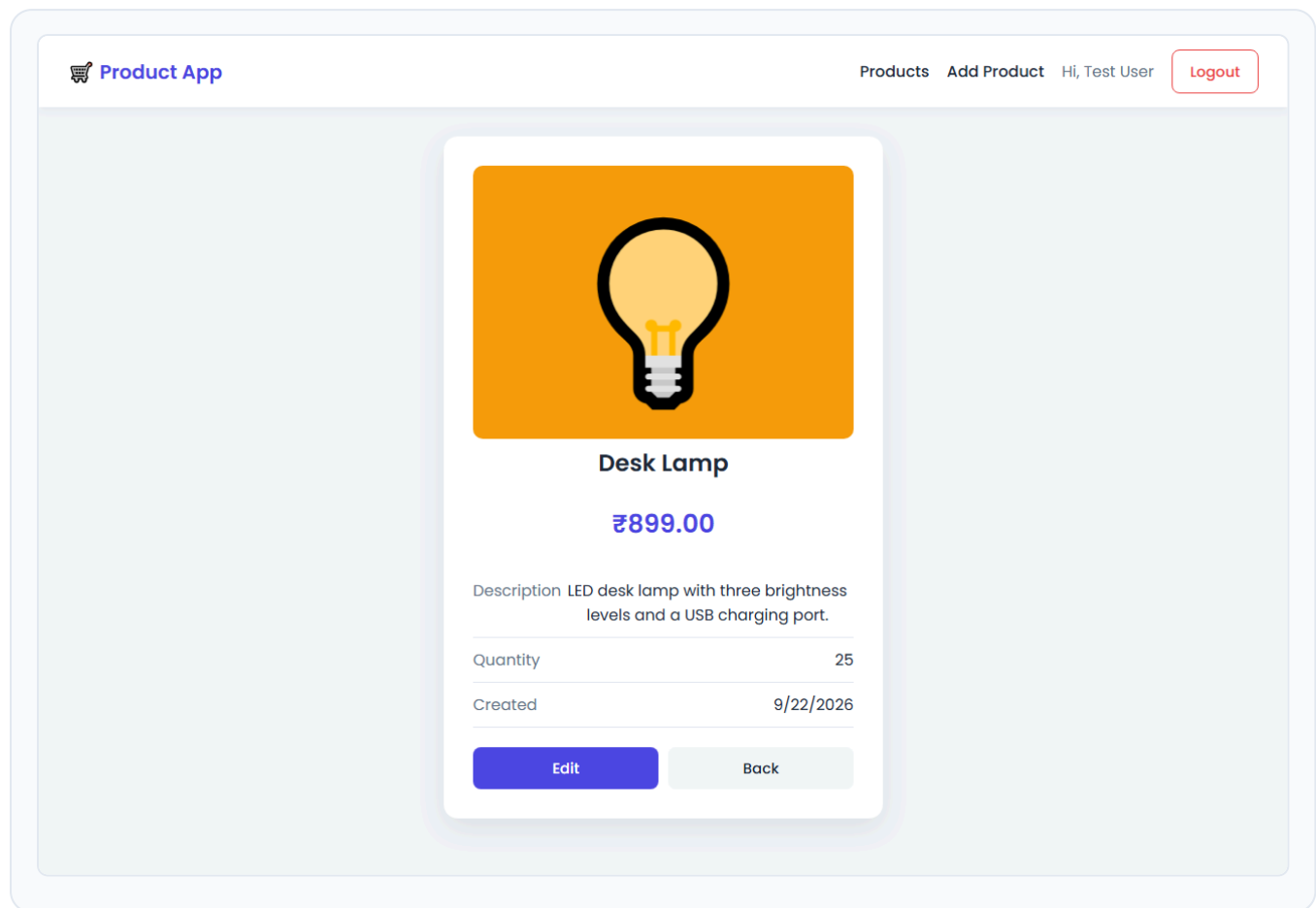
What it does

- Loads the product with `GET /api/products/:id`
- Saves with `PUT /api/products/:id`
- Shows the current picture under the file box, so you can see what is there

About the picture: leaving the file box empty keeps the old picture. Choosing a new file replaces it.

File: `frontend/src/pages/products/ProductForm.jsx` (same file as Add)

Page 6 — Product View



A read-only card showing one product: big picture, name, price, description, quantity and the date it was created.

What it does

- Loads one product with `GET /api/products/:id`
- **Edit** goes to the edit form
- **Back** returns to the list

Nothing can be changed here. It is only for looking.

File: `frontend/src/pages/products/ProductView.jsx`

API reference

Method	URL	Login needed	What it does
POST	/api/auth/register	No	Make an account
POST	/api/auth/login	No	Log in, returns a token
GET	/api/products?page=&search=	Yes	List products, 5 per page
GET	/api/products/:id	Yes	Get one product
POST	/api/products	Yes	Create a product
PUT	/api/products/:id	Yes	Update a product
DELETE	/api/products/:id	Yes	Delete a product

Protected routes need this header:

```
Authorization: Bearer <token>
```

Without it the server replies `401 {"message":"Please login first"} .`

Database tables

users

Column	Type	Notes
id	INT	Primary key, auto
name	VARCHAR(100)	
email	VARCHAR(100)	Unique
password	VARCHAR(255)	bcrypt hash, never plain text
created_at	TIMESTAMP	

products

Column	Type	Notes
id	INT	Primary key, auto
name	VARCHAR(100)	
description	TEXT	Can be empty
price	DECIMAL(10,2)	
quantity	INT	
image	VARCHAR(255)	File name only, can be empty
user_id	INT	Links to users.id
created_at	TIMESTAMP	

The `image` column stores only the file name. The full address is built in the frontend as `http://localhost:5000/uploads/` + file name.